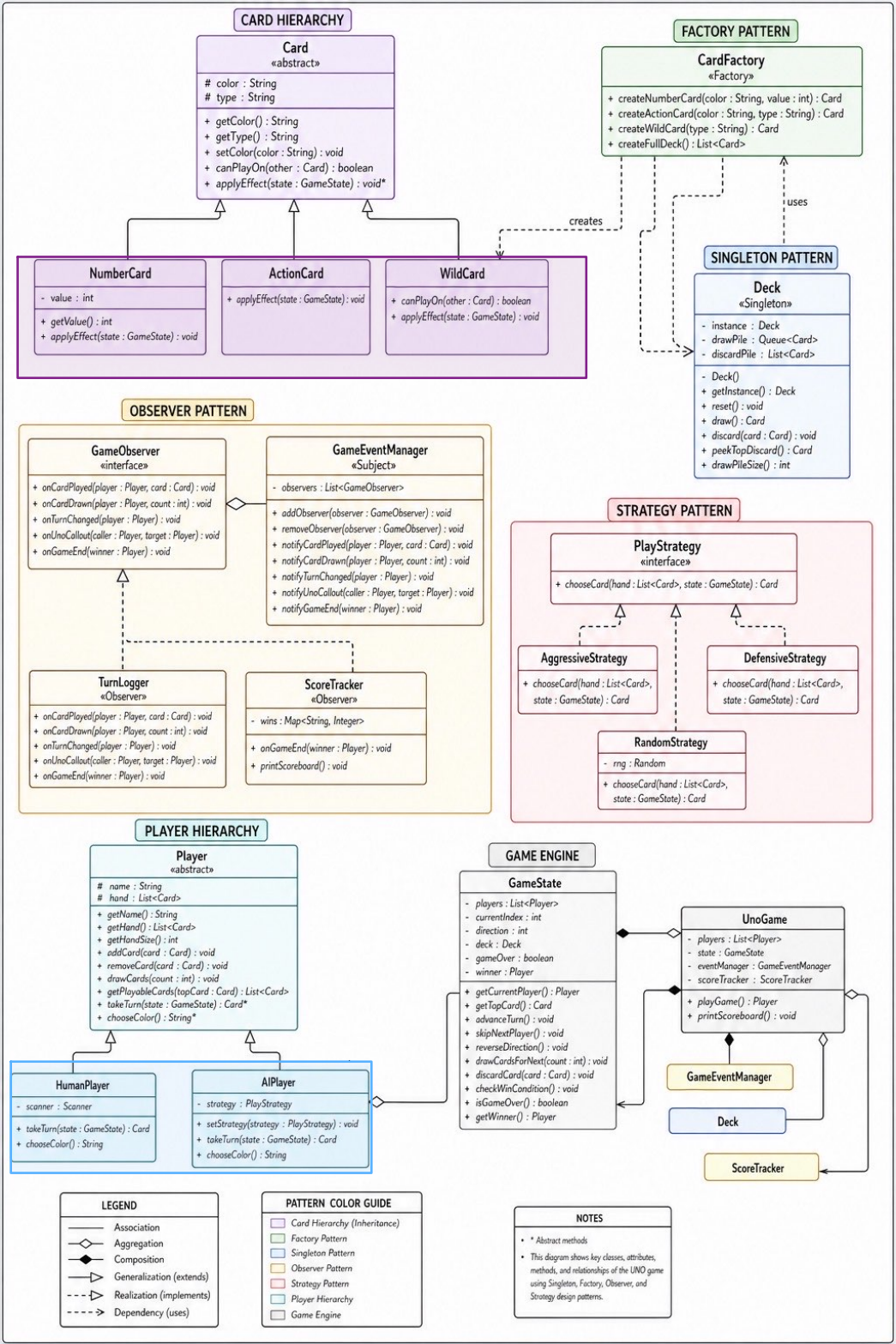
CARD HIERARCHY
Card
«abstract»
color : String
type : String
+ getColor() : String
+ getType() : String
+ setColor(color : String) : void
+ canPlayOn(other : Card) : boolean
+ applyEffect(state : GameState) : void*
NumberCard
- value : int
+ getValue() : int
+ applyEffect(state : GameState) : void
ActionCard
+ applyEffect(state : GameState) : void
WildCard
+ canPlayOn(other : Card) : boolean
+ applyEffect(state : GameState) : void
FACTORY PATTERN
CardFactory
«Factory»
+ createNumberCard(color : String, value : int) : Card
+ createActionCard(color : String, type : String) : Card
+ createWildCard(type : String) : Card
+ createFullDeck() : List<Card>
creates
uses
SINGLETON PATTERN
Deck
«Singleton»
- instance : Deck
- drawPile : Queue<Card>
- discardPile : List<Card>
- Deck()
+ getInstance() : Deck
+ reset() : void
+ draw() : Card
+ discard(card : Card) : void
+ peekTopDiscard() : Card
+ drawPileSize() : int
OBSERVER PATTERN
GameObserver
«interface»
+ onCardPlayed(player : Player, card : Card) : void
+ onCardDrawn(player : Player, count : int) : void
+ onTurnChanged(player : Player) : void
+ onUnoCallout(caller : Player, target : Player) : void
+ onGameEnd(winner : Player) : void
GameEventManager
«Subject»
- observers : List<GameObserver>
+ addObserver(observer : GameObserver) : void
+ removeObserver(observer : GameObserver) : void
+ notifyCardPlayed(player : Player, card : Card) : void
+ notifyCardDrawn(player : Player, count : int) : void
+ notifyTurnChanged(player : Player) : void
+ notifyUnoCallout(caller : Player, target : Player) : void
+ notifyGameEnd(winner : Player) : void
TurnLogger
«Observer»
+ onCardPlayed(player : Player, card : Card) : void
+ onCardDrawn(player : Player, count : int) : void
+ onTurnChanged(player : Player) : void
+ onUnoCallout(caller : Player, target : Player) : void
+ onGameEnd(winner : Player) : void
ScoreTracker
«Observer»
- wins : Map<String, Integer>
+ onGameEnd(winner : Player) : void
+ printScoreboard() : void
STRATEGY PATTERN
PlayStrategy
«interface»
+ chooseCard(hand : List<Card>, state : GameState) : Card
AggressiveStrategy
+ chooseCard(hand : List<Card>, state : GameState) : Card
DefensiveStrategy
+ chooseCard(hand : List<Card>, state : GameState) : Card
RandomStrategy
- rng : Random
+ chooseCard(hand : List<Card>, state : GameState) : Card
PLAYER HIERARCHY
Player
«abstract»
name : String
hand : List<Card>
+ getName() : String
+ getHand() : List<Card>
+ getHandSize() : int
+ addCard(card : Card) : void
+ removeCard(card : Card) : void
+ drawCards(count : int) : void
+ getPlayableCards(topCard : Card) : List<Card>
+ takeTurn(state : GameState) : Card*
+ chooseColor() : String*
HumanPlayer
- scanner : Scanner
+ takeTurn(state : GameState) : Card
+ chooseColor() : String
AIPlayer
- strategy : PlayStrategy
+ setStrategy(strategy : PlayStrategy) : void
+ takeTurn(state : GameState) : Card
+ chooseColor() : String
GAME ENGINE
GameState
- players : List<Player>
- currentIndex : int
- direction : int
- deck : Deck
- gameOver : boolean
- winner : Player
+ getCurrentPlayer() : Player
+ getTopCard() : Card
+ advanceTurn() : void
+ skipNextPlayer() : void
+ reverseDirection() : void
+ drawCardsForNext(count : int) : void
+ discardCard(card : Card) : void
+ checkWinCondition() : void
+ isGameOver() : boolean
+ getWinner() : Player
UnoGame
- players : List<Player>
- state : GameState
- eventManager : GameEventManager
- scoreTracker : ScoreTracker
+ playGame() : Player
+ printScoreboard() : void
GameEventManager
Deck
ScoreTracker
LEGEND
Association
Aggregation
Composition
Generalization (extends)
Realization (implements)
Dependency (uses)
PATTERN COLOR GUIDE
Card Hierarchy (Inheritance)
Factory Pattern
Singleton Pattern
Observer Pattern
Strategy Pattern
Player Hierarchy
Game Engine
NOTES
* Abstract methods
This diagram shows key classes, attributes, methods, and relationships of the UNO game using Singleton, Factory, Observer, and Strategy design patterns.